

UNIVERSIDAD NACIONAL DE MOQUEGUA



INGENIERÍA DE SISTEMAS E INFORMATICA

Estudiantes:

Leo Enrique Pari Puma

Rousse Daniela Martinez Briceño

Franco Enrique Ivo Santi Colque

Giovanni Bierhorf Ccallata Ancori

Materia:

Calidad de Software

Docente:

Juan Carlos Valero Gomez

Ciclo y Sección:

VII Ciclo - Sección "A"

Ilo - Perú

2026

Contenido

Definición y Alcance del Tema	4
Introducción a las Pruebas de Contrato	4
Definición de las Pruebas de Contrato	4
Objetivo de las Pruebas de Contrato	5
Defectos que permiten detectar	6
Beneficios e Importancia	8
Base Normativa y Estándares	8
Fundamento Normativo Internacional	8
ISO/IEC 25010:2023 — Modelo de Calidad del Producto Software (SQuaRE).....	8
ISO/IEC/IEEE 29119 — Software Testing	9
ISO/IEC/IEEE 12207:2017 — Software Life Cycle Processes	10
OWASP API Security Top 10 (2023).....	10
Tabla Comparativa Normativa.....	11
Relación con el Marco ISTQB.....	11
Contract Testing en el Ciclo de Vida del Software	12
Integración con el SDLC y el Enfoque Shift-Left Testing	12
Continuous Testing y Pipelines CI/CD.....	13
DevOps y la Cultura de Calidad Compartida	13
Metodología de Ejecución	13
¿Quién la realiza? (Roles y Responsabilidades)	13
¿En qué fase del SDLC se aplica?.....	14
¿Cómo se diseñan los casos de prueba?	14
Ecosistema de Herramientas por Lenguaje	15
JavaScript / TypeScript	16

Jest.....	16
Mocha.....	17
Python.....	17
PyTest.....	17
Unittest	18
PHP	18
PHPUnit.....	18
Pest	18
Java	19
JUnit	19
TestNG	19
Selenium.....	20
C# / .NET.....	20
NUnit.....	20
xUnit.....	20
MSTest	21
Estrategia de Automatización.....	21
Automatización mediante CI/CD (Pipelines).....	21
¿Qué ocurre cuando una prueba falla y cómo se gestionan esas fallas?.....	22
Conclusiones	23
Bibliografía	23

Definición y Alcance del Tema

Introducción a las Pruebas de Contrato

El crecimiento de las arquitecturas basadas en microservicios, APIs y sistemas distribuidos ha permitido desarrollar aplicaciones más escalables y flexibles; sin embargo, también ha incrementado la complejidad de la comunicación entre servicios independientes. En este contexto, surgió la necesidad de contar con mecanismos que permitieran detectar de manera temprana los problemas de compatibilidad ocasionados por cambios en las interfaces de comunicación, evitando errores durante la integración y el despliegue de nuevas versiones (Fowler, 2014).

Como respuesta a esta necesidad, las pruebas de contrato se consolidaron como una estrategia para validar la compatibilidad entre los servicios antes de su integración. Actualmente, forman parte de las prácticas recomendadas en el desarrollo de software moderno, especialmente en metodologías ágiles y procesos DevOps, ya que contribuyen a mejorar la calidad del software, facilitar el trabajo entre equipos independientes y fortalecer los procesos de integración y entrega continua (Pact Foundation, 2025).

Definición de las Pruebas de Contrato

Las Pruebas de Contrato (Contract Testing) son un tipo de prueba de software que verifica que la comunicación entre dos componentes de un sistema, generalmente un consumidor (Consumer) y un proveedor (Provider), cumpla con un conjunto de especificaciones previamente acordadas, conocido como contrato. Este contrato establece las reglas que deben respetarse durante la interacción entre ambos servicios, garantizando que la información intercambiada sea compatible y consistente (Pact Foundation, 2025).

De acuerdo con Fowler (2011), un contrato describe el comportamiento esperado de una interfaz de comunicación, especificando aspectos como los endpoints disponibles, los métodos HTTP permitidos (GET, POST, PUT, DELETE, entre otros), los parámetros de entrada, la estructura y el formato de los datos intercambiados (por ejemplo, JSON o XML), así como los códigos de estado HTTP

que el proveedor debe devolver ante cada solicitud. Las pruebas de contrato verifican automáticamente que dichas especificaciones se mantengan sin modificaciones incompatibles.

A diferencia de las pruebas unitarias, que validan componentes individuales, o de las pruebas de integración, que requieren la interacción entre varios sistemas en ejecución, las pruebas de contrato se enfocan exclusivamente en comprobar el cumplimiento de las interfaces de comunicación. Esto permite detectar cambios incompatibles entre consumidores y proveedores antes de integrar completamente los servicios, reduciendo los riesgos durante el desarrollo y el despliegue de nuevas versiones (Richardson, 2021).

Gracias a este enfoque, las pruebas de contrato favorecen el desarrollo independiente de los servicios, disminuyen los problemas de integración y fortalecen la confiabilidad de las aplicaciones distribuidas, convirtiéndose en una práctica ampliamente adoptada en arquitecturas de microservicios y en procesos de Integración Continua y Entrega Continua (CI/CD) (Richardson, 2021).

Objetivo de las Pruebas de Contrato

El objetivo principal de las Pruebas de Contrato (Contract Testing) es garantizar que la comunicación entre un consumidor y un proveedor de servicios se mantenga consistente y compatible, verificando que ambas partes cumplan con las especificaciones definidas en el contrato. De esta manera, se asegura que los cambios realizados en un servicio no afecten el correcto funcionamiento de las aplicaciones o sistemas que dependen de él (Pact Foundation, 2025).

Estas pruebas buscan detectar de forma temprana incompatibilidades relacionadas con la estructura de los datos, los métodos de comunicación, los parámetros de entrada y salida, los códigos de respuesta HTTP y cualquier modificación que pueda romper el acuerdo establecido entre los servicios. Al identificar estos problemas antes de la integración o el despliegue, se reduce significativamente el riesgo de errores en los entornos de producción (Fowler, 2014).

Asimismo, las Pruebas de Contrato permiten que diferentes equipos de desarrollo trabajen de manera independiente sobre sus respectivos servicios sin necesidad de depender continuamente de otros componentes del sistema. Esto favorece el desarrollo paralelo, acelera los procesos de integración

continua (CI) y entrega continua (CD), y contribuye a mejorar la calidad, confiabilidad y mantenibilidad del software (International Software Testing Qualification Board, 2023).

En resumen, el objetivo de las Pruebas de Contrato no es evaluar la lógica interna de una aplicación, sino garantizar que la interfaz de comunicación entre los servicios permanezca estable, permitiendo una integración segura y reduciendo los costos asociados a errores de compatibilidad (Pact Foundation, 2025; Fowler, 2011).

Defectos que permiten detectar

Entre los principales defectos que permiten detectar las pruebas de contrato se encuentran:

- Cambios en la estructura de los datos: Ocurren cuando el proveedor modifica la estructura del mensaje, agregando, eliminando o renombrando atributos que el consumidor espera recibir. Estas modificaciones pueden impedir que el consumidor interprete correctamente la información intercambiada (Pact Foundation, 2025).
- Incompatibilidad de tipos de datos: Se presenta cuando un atributo cambia su tipo de dato, por ejemplo, de un valor numérico a una cadena de texto, provocando errores durante el procesamiento o validación de la información (Fowler, 2011).
- Modificación de endpoints o rutas: Consiste en cambios en la URL o en la ruta de acceso definida en el contrato, lo que ocasiona que el consumidor no pueda localizar el recurso solicitado (Pact Foundation, 2025).
- Cambios en los métodos HTTP: Se detectan cuando el proveedor modifica el método HTTP asociado a una operación (como GET, POST, PUT o DELETE), generando incompatibilidades con las solicitudes realizadas por el consumidor (Pact Foundation, 2025).
- Alteración de los códigos de estado HTTP: Ocurre cuando el proveedor devuelve códigos de respuesta diferentes a los establecidos en el contrato, afectando el comportamiento esperado por la aplicación consumidora (Fowler, 2011).

- Eliminación o modificación de parámetros: Se produce cuando se eliminan parámetros obligatorios o se modifica su significado, ocasionando solicitudes inválidas o respuestas incorrectas durante la comunicación entre los servicios (Richardson, 2021).
- Inconsistencias en el formato de intercambio de datos: Se presentan cuando existen diferencias en el formato acordado para el intercambio de información, como JSON o XML, impidiendo que el consumidor procese correctamente la respuesta enviada por el proveedor (Pact Foundation, 2025).

La detección temprana de estos defectos permite corregir las incompatibilidades antes de la integración o el despliegue de nuevas versiones, disminuyendo el riesgo de fallos en producción y contribuyendo a mantener la estabilidad de las aplicaciones distribuidas (ISTQB, 2023).

Tabla 1. Principales defectos detectados mediante Pruebas de Contrato

Defecto detectado	Descripción
Cambio en la estructura de datos	Se agregan, eliminan o renombran atributos definidos en el contrato, afectando la interpretación de la información por parte del consumidor.
Cambio de tipo de datos	Un campo cambia de tipo (por ejemplo, de entero a cadena de texto), provocando errores en el procesamiento de los datos.
Modificación de endpoints	Se altera la ruta o URL utilizada para acceder al servicio, impidiendo que el consumidor encuentre el recurso esperado.
Cambio de métodos HTTP	Se modifica el método permitido para una operación (GET, POST, PUT o DELETE), generando incompatibilidades en la comunicación.
Alteración de códigos HTTP	El servicio devuelve códigos de respuesta distintos a los establecidos en el contrato, afectando el comportamiento esperado del consumidor.
Modificación de parámetros	Se eliminan, agregan o cambian parámetros requeridos por el contrato, ocasionando solicitudes inválidas o respuestas incorrectas.
Inconsistencia en formatos	Se producen cambios en la estructura o formato de los mensajes intercambiados (JSON o XML), dificultando la correcta interpretación de los datos.

Fuente: Elaboración Propia}

Beneficios e Importancia

Uno de sus principales beneficios es la detección temprana de incompatibilidades entre el consumidor y el proveedor de un servicio, evitando que cambios en la interfaz de comunicación generen fallos durante la integración o en los entornos de producción. Esto disminuye el tiempo dedicado a la corrección de errores y reduce los costos asociados al mantenimiento del software (Pact Foundation, 2025).

Asimismo, las Pruebas de Contrato permiten que diferentes equipos de desarrollo trabajen de manera independiente sobre sus propios servicios, sin necesidad de esperar que todo el sistema esté disponible para realizar validaciones. Esta característica favorece el desarrollo paralelo, mejora la colaboración entre equipos y agiliza los procesos de integración continua (CI) y entrega continua (CD) (Fowler, 2011).

Otro aspecto importante es que proporcionan mayor confianza al momento de desplegar nuevas versiones del software, ya que verifican automáticamente que los contratos establecidos entre los servicios continúen cumpliéndose. Como resultado, se incrementa la estabilidad de las aplicaciones y se reduce la probabilidad de que un cambio afecte funcionalidades ya existentes (Pact Foundation, 2025; ISTQB, 2023).

Base Normativa y Estándares

Fundamento Normativo Internacional

A continuación, se presenta el análisis de los principales estándares y marcos internacionales que respaldan, de forma directa o indirecta, la práctica de las Pruebas de Contrato, detallando el objetivo de cada norma, su relación con el Contract Testing y los beneficios que aporta a la calidad del software.

ISO/IEC 25010:2023 — Modelo de Calidad del Producto Software (SQuaRE)

La ISO/IEC 25010:2023, perteneciente a la familia SQuaRE, define el modelo de referencia para evaluar la calidad de productos de software y sistemas TIC. Establece nueve características de

calidad, entre ellas Aptitud Funcional, Compatibilidad, Fiabilidad, Seguridad y Mantenibilidad, sirviendo como referente internacional para especificar y evaluar la calidad del software.

Las Pruebas de Contrato contribuyen principalmente a la Compatibilidad, al verificar la interoperabilidad entre servicios, y a la Aptitud Funcional, asegurando que el proveedor cumpla con las funciones esperadas por el consumidor. Además, fortalecen la Fiabilidad, al detectar incompatibilidades antes de producción, y la Mantenibilidad, ya que los contratos actúan como documentación ejecutable de las interfaces.

ISO/IEC 25010:2023 provee el vocabulario normativo que permite a los equipos de desarrollo y QA justificar, planificar y medir el impacto de las Pruebas de Contrato en términos reconocidos por la industria. Al mapear los resultados de los tests de contrato sobre las subcaracterísticas del estándar, los líderes técnicos pueden cuantificar mejoras en la calidad del producto e incluirlas en informes de gestión con respaldo normativo internacional.

ISO/IEC/IEEE 29119 — Software Testing

La serie ISO/IEC/IEEE 29119 es el principal estándar internacional para pruebas de software. Establece conceptos, procesos, documentación y técnicas de prueba aplicables a cualquier organización y ciclo de vida, incluyendo metodologías ágiles y DevOps. Su finalidad es proporcionar un marco para planificar, ejecutar y documentar las pruebas de manera consistente y trazable.

Las Pruebas de Contrato se alinean con el enfoque de pruebas basado en riesgos definido por la norma, al reducir los riesgos de integración entre servicios. Además, se ubican en el nivel de pruebas de integración y utilizan técnicas de validación de interfaces, donde los contratos representan casos de prueba derivados de las especificaciones de las APIs.

La serie ISO/IEC/IEEE 29119 proporciona el marco de procesos bajo el cual las Pruebas de Contrato pueden ser planificadas, ejecutadas y reportadas de manera formal y trazable. Esto es especialmente valioso en contextos regulados o en proyectos donde se requiere demostrar conformidad con estándares internacionales de prueba.

ISO/IEC/IEEE 12207:2017 — Software Life Cycle Processes

La ISO/IEC/IEEE 12207:2017 establece un marco para los procesos del ciclo de vida del software, incluyendo desarrollo, operación, mantenimiento y procesos de soporte como verificación, validación, gestión de la configuración y aseguramiento de la calidad. Es el estándar de referencia para gestionar el ciclo de vida del software.

Las Pruebas de Contrato se vinculan principalmente con los procesos de Verificación y Validación, ya que comprueban que el proveedor cumpla el contrato acordado y que el consumidor pueda utilizar correctamente el servicio. Asimismo, la Gestión de la Configuración respalda el control y versionado de los contratos, mientras que el proceso de Integración del Software se beneficia del Contract Testing para validar las interacciones entre servicios de forma temprana y continua.

El estándar también contempla el proceso de Integración del Software, que en el contexto de arquitecturas de microservicios corresponde precisamente a la validación de las interacciones entre servicios. Las Pruebas de Contrato constituyen la técnica más eficiente para automatizar este proceso de integración de manera temprana y continua, en consonancia con los principios de Shift-Left Testing alineados con la norma.

ISO/IEC/IEEE 12207:2017 proporciona el contexto de ciclo de vida en el que el Contract Testing adquiere su valor pleno: al situarse en la fase de integración del software, actúa como una barrera preventiva que impide que incompatibilidades de interfaz propaguen defectos hacia fases más costosas como las pruebas de sistema o las pruebas de aceptación.

OWASP API Security Top 10 (2023)

El OWASP API Security Top 10 (2023) identifica las principales vulnerabilidades que afectan a las APIs modernas, proporcionando buenas prácticas para su diseño, prueba y protección. Es un referente internacional en seguridad de APIs (OWASP, 2023).

Aunque las Pruebas de Contrato no son un mecanismo de seguridad, complementan la protección de las APIs al verificar que consumidores y proveedores cumplan el contrato establecido. Esto ayuda a prevenir la exposición de datos no previstos (API3: BOPLA), facilita el control de

versiones y el inventario de APIs (API9) y asegura que las respuestas de APIs externas se ajusten al contrato definido (API10) (SecureCodingHub, 2026).

El Contract Testing reduce errores derivados de cambios inesperados en las interfaces y mantiene la coherencia entre la implementación y la documentación de las APIs, fortaleciendo su confiabilidad y seguridad.

Tabla Comparativa Normativa

Tabla 2. Comparación de principales estándares y marcos relacionados con Contract Testing

Norma / Marco	Objetivo Principal	Relación con Contract Testing	Beneficio Clave
ISO/IEC 25010:2023	Modelo de calidad del producto software	Provee los atributos de calidad que el Contract Testing mejora: Compatibilidad, Fiabilidad, Mantenibilidad y Aptitud Funcional	Marco normativo para justificar y medir el impacto del CT en la calidad del producto
ISO/IEC/IEEE 29119	Estándar internacional de testing de software	Define los procesos, técnicas y documentación bajo los cuales el CT puede ser planificado, ejecutado y reportado formalmente	Marco de procesos y documentación para la gestión formal del CT en proyectos de software
ISO/IEC/IEEE 12207:2017	Procesos del ciclo de vida del software	Sitúa el CT en los procesos de Verificación e Integración del Software, alineado con el Shift-Left Testing	Contexto de ciclo de vida y gestión de configuración para los contratos de interfaz
OWASP API Security Top 10 (2023)	Marco de seguridad de APIs	El CT contribuye a mitigar BOPLA y el problema de las shadow APIs; complementa la seguridad de las interfaces	Reducción de la superficie de ataque derivada de comportamientos no documentados de APIs
ISTQB CTFL v4.0	Certificación y conocimiento en testing	El CT se ubica en el nivel de pruebas de integración de componentes y respalda el principio de Shift-Left Testing	Marco de conocimiento y certificación profesional que reconoce el CT como práctica esencial

Fuente: Elaboración Propia

Nota. Elaboración propia a partir del análisis de ISO/IEC 25010:2023, ISO/IEC/IEEE 29119, ISO/IEC/IEEE 12207:2017, ISTQB CTFL v4.0 (2023) y OWASP API Security Top 10 (2023).

Relación con el Marco ISTQB

El International Software Testing Qualifications Board (ISTQB) es la principal organización internacional dedicada a la certificación y estandarización del conocimiento en pruebas de software. Su

Certified Tester Foundation Level v4.0 (CTFL 4.0) reconoce metodologías como Agile, DevOps y Continuous Delivery como contextos adecuados para la aplicación de técnicas de prueba (International Software Testing Qualification Board, 2023).

Según el ISTQB, las Pruebas de Contrato se ubican principalmente en el nivel de pruebas de integración de componentes, ya que verifican las interfaces y la comunicación entre servicios. En arquitecturas de microservicios, también pueden aplicarse a nivel de integración de sistemas cuando intervienen plataformas o servicios externos.

Esta práctica se alinea con varios principios fundamentales del testing: demuestra la presencia de defectos, prioriza las áreas de mayor riesgo, evita pruebas exhaustivas al centrarse en las interacciones realmente utilizadas por el consumidor y favorece el Shift Left Testing, detectando incompatibilidades de forma temprana para reducir tiempo y costos.

El Contract Testing complementa las pruebas de integración al validar las interfaces entre componentes o sistemas, garantizando la compatibilidad de las APIs antes de la integración completa.

Contract Testing en el Ciclo de Vida del Software

El Contract Testing forma parte del ciclo de vida del desarrollo de software (SDLC) y se integra con metodologías modernas como Agile, DevOps, Continuous Testing y arquitecturas de microservicios, permitiendo validar la compatibilidad entre servicios desde las primeras etapas del desarrollo.

Integración con el SDLC y el Enfoque Shift-Left Testing

El Shift-Left Testing es una práctica metodológica, fundamentada en el principio de testing temprano del ISTQB y respaldada por ISO/IEC/IEEE 12207:2017 en su proceso de verificación, que consiste en adelantar las actividades de prueba hacia las fases más tempranas del SDLC, reduciendo el costo de la detección y corrección de defectos. Como han documentado Fowler & Clemson (2014) en su análisis de estrategias de testing para microservicios, los tests de alto nivel, como los tests de extremo a extremo, son lentos, frágiles y costosos de mantener; en cambio, los tests de bajo nivel, como los tests unitarios y los tests de contrato, son rápidos, confiables y altamente informativos.

Las Pruebas de Contrato siguen este enfoque al ejecutarse junto con las pruebas unitarias del consumidor y del proveedor, validando los contratos sin necesidad de desplegar todos los servicios ni contar con un entorno de integración completo

Continuous Testing y Pipelines CI/CD

El Continuous Testing consiste en ejecutar pruebas de forma automática durante todo el pipeline de CI/CD. En este proceso, las Pruebas de Contrato generan y verifican los contratos entre consumidores y proveedores; si la validación es exitosa, el despliegue continúa, mientras que cualquier incompatibilidad bloquea el proceso y notifica al equipo responsable. Esto permite detectar errores de integración en minutos en lugar de hacerlo en etapas posteriores.

DevOps y la Cultura de Calidad Compartida

El Contract Testing fomenta la colaboración entre consumidores y proveedores al definir y mantener contratos de manera conjunta. Esta práctica se alinea con los principios de DevOps y el modelo CALMS, promoviendo la comunicación entre equipos y el uso de contratos como documentación ejecutable y siempre actualizada.

Metodología de Ejecución

Esta sección detalla los aspectos operativos, responsabilidades y el diseño de los escenarios de prueba en el ciclo de vida del software.

¿Quién la realiza? (Roles y Responsabilidades)

Las pruebas de contrato requieren una colaboración activa entre tres roles principales del equipo de ingeniería (Pactflow, 2024):

- **Desarrolladores (Devs):** Son los ejecutores principales y encargados de escribir el código de las pruebas. El desarrollador del Consumidor (ej. Frontend o App Móvil) define la estructura de datos que necesita consumir, generando el archivo de contrato. Por otro lado, el desarrollador del Proveedor (ej. Backend o API) programa la lógica y

los estados necesarios para validar que su servicio cumpla de manera estricta con dicho acuerdo.

- Ingenieros de Aseguramiento de la Calidad (QA / Testers): Actúan como estrategias y auditores del proceso. Su función es identificar los puntos críticos de integración entre los sistemas, definir qué flujos de datos deben estar respaldados bajo un contrato y asegurar la cobertura de escenarios tanto de éxito como de manejo de errores (ISTQB, 2024).
- Ingenieros DevOps: Son los facilitadores de la infraestructura tecnológica. Se encargan de configurar y mantener el servidor centralizado donde se almacenan y consultan los contratos (como Pact Broker), además de automatizar la ejecución de los scripts de prueba dentro de los entornos compartidos.

¿En qué fase del SDLC se aplica?

Estas pruebas se aplican bajo el enfoque "*Shift-Left*" (Pruebas Tempranas), ubicándose estrictamente en la Fase de Codificación y Pruebas Unitarias dentro del Ciclo de Vida del Desarrollo de Software o SDLC (International Software Testing Qualification Board, 2023).

A diferencia de las pruebas de integración tradicionales o *End-to-End* (E2E), las pruebas de contrato no requieren que los sistemas estén completamente desplegados en un servidor ni conectados a redes o bases de datos reales. Se ejecutan de manera aislada en la máquina local del desarrollador en cuestión de segundos, lo que permite detectar incompatibilidades en las interfaces de comunicación antes de compilar, empaquetar o subir el código a los repositorios comunes (Fowler, 2014).

¿Cómo se diseñan los casos de prueba?

El diseño de los casos de prueba se basa en la definición formal de interacciones individuales que ocurren cuando dos sistemas intentan comunicarse. Para construir cada escenario de prueba, se deben estructurar obligatoriamente tres componentes teóricos que definen las reglas del intercambio de información (Fowler, 2011; Micallef, 2021):

- La Precondición del Servidor (Estado del Proveedor): Consiste en definir el escenario o la situación previa en la que debe estar el sistema que recibe la solicitud para que la prueba tenga sentido. En este paso se describe qué información o qué configuraciones internas debe tener listas el servidor antes de que el cliente intente comunicarse con él.
- La Petición (Mensaje de Entrada): Describe con total precisión la acción que va a ejecutar el sistema cliente. En este componente se especifica la ruta de acceso que se va a utilizar, la acción técnica que se desea realizar y las credenciales o permisos de seguridad obligatorios que deben viajar en la comunicación para que el mensaje sea aceptado.
- La Respuesta Esperada (Mensaje de Salida): Determina el acuerdo de formato y la estructura que el servidor está obligado a devolver para que el cliente pueda procesar la información correctamente. Aquí se establece el tipo de respuesta que confirmará si la operación fue exitosa o fallida, y la organización que deben tener los datos devueltos para evitar que el sistema cliente falle.

Ecosistema de Herramientas por Lenguaje

Las pruebas de contrato (Contract Testing) se han consolidado como una práctica esencial para garantizar la interoperabilidad entre servicios en arquitecturas distribuidas y de microservicios. A diferencia de las pruebas de integración tradicionales, este enfoque verifica que las interfaces de comunicación entre consumidores y proveedores cumplan con un contrato previamente definido, permitiendo detectar incompatibilidades antes de que los sistemas sean desplegados o integrados en un entorno de producción. Según Fowler (2014), las pruebas de contrato reducen el riesgo de fallos durante la integración al validar que ambas partes respeten las expectativas establecidas para el intercambio de datos (Fowler, 2014).

La implementación efectiva de las pruebas de contrato depende de un ecosistema de herramientas que facilite la creación, gestión y verificación de dichos contratos. En la actualidad, el framework Pact se ha convertido en una de las soluciones más adoptadas para este propósito, ofreciendo implementaciones específicas para diversos lenguajes de programación, como JavaScript/TypeScript,

Python, PHP, Java y C#/.NET. El enfoque Consumer-Driven Contract Testing (CDCT) promovido por Pact permite que los consumidores definan las expectativas sobre una API, mientras que los proveedores verifican automáticamente el cumplimiento de esos contratos, favoreciendo el desarrollo independiente de los servicios y reduciendo los errores de integración (Holshausen, 2020).

Este ecosistema proporciona implementaciones oficiales para diferentes lenguajes, entre ellas Pact JS (JavaScript/TypeScript), Pact Python, Pact PHP, Pact JVM (Java) y PactNet (C#/.NET). Aunque cada implementación está adaptada a las particularidades de su respectivo lenguaje, todas comparten la misma arquitectura, el enfoque Consumer-Driven Contract Testing (CDCT) y funcionalidades equivalentes, como la generación automática de contratos en formato JSON, la verificación de contratos entre consumidores y proveedores, la integración con Pact Broker y la compatibilidad con procesos de Integración Continua y Entrega Continua (CI/CD). La principal diferencia radica en el framework de pruebas con el que se integran, como Jest o Mocha en JavaScript, PyTest o unittest en Python, PHPUnit o Pest en PHP, JUnit o TestNG en Java, y NUnit, xUnit o MSTest en C#/.NET, los cuales proporcionan el entorno de ejecución donde se desarrollan y ejecutan las pruebas de contrato (Holshausen, 2020) (Pact Foundation, 2025).

JavaScript / TypeScript

Las pruebas de contrato desempeñan un papel fundamental al verificar que la comunicación entre consumidores y proveedores mantenga un comportamiento consistente conforme evolucionan los servicios. De acuerdo con Fowler (2014), las pruebas de contrato permiten validar las expectativas existentes entre componentes independientes, reduciendo el riesgo de fallos durante la integración y disminuyendo la necesidad de ejecutar pruebas End-to-End para cada modificación del sistema (Fowler, 2014).

Jest

Jest proporciona un entorno de pruebas rápido, soporte para pruebas asincrónicas, generación automática de reportes y una integración sencilla con proyectos Node.js y TypeScript (Meta Open Source, 2025).

Aunque Jest no es una herramienta de Contract Testing por sí misma, constituye uno de los frameworks de pruebas más utilizados para ejecutar las pruebas que generan contratos mediante Pact JS.

Mocha

Es un framework de pruebas para Node.js compatible con Pact JS. Según la documentación oficial de Mocha (2025), esta herramienta ofrece gran flexibilidad para construir pruebas asincrónicas, organizar suites de pruebas e integrarse con diferentes bibliotecas de aserciones como Chai (Mocha.js, 2025).

Mocha suele emplearse en proyectos que requieren una mayor personalización del entorno de pruebas o que ya utilizan este framework dentro de su infraestructura de desarrollo.

Python

En arquitecturas basadas en microservicios, las pruebas de contrato permiten garantizar que la comunicación entre consumidores y proveedores permanezca estable a pesar de la evolución independiente de cada servicio. Según Richardson (2021), este tipo de pruebas constituye una práctica esencial para minimizar errores de integración y asegurar la compatibilidad entre servicios sin depender exclusivamente de pruebas End-to-End (Richardson, 2021).

PyTest

PyTest es el framework de pruebas más utilizado en Python y suele emplearse para ejecutar las pruebas que generan contratos mediante Pact Python. De acuerdo con Okken (2022), PyTest proporciona una sintaxis sencilla, soporte para pruebas parametrizadas, fixtures reutilizables y una amplia variedad de complementos (plugins), características que favorecen la automatización de pruebas en proyectos de cualquier tamaño (Okken, 2022).

Unittest

Es el framework de pruebas incluido en la biblioteca estándar de Python. Basado en la arquitectura xUnit, permite organizar casos de prueba, ejecutar verificaciones automáticas y generar reportes de resultados. Unittest puede utilizarse junto con Pact Python para construir y ejecutar pruebas de contrato, especialmente en proyectos que utilizan únicamente herramientas estándar del lenguaje (Python Software Foundation, 2025).

PHP

En el ecosistema PHP, la implementación de Contract Testing se realiza principalmente mediante Pact PHP, una implementación oficial del framework Pact que sigue el enfoque Consumer-Driven Contract Testing (CDCT). De acuerdo con Holshausen (2020), este enfoque permite que el consumidor defina las expectativas de interacción con el proveedor mediante contratos ejecutables, los cuales son posteriormente verificados para asegurar la compatibilidad entre ambos servicios (Holshausen, 2020).

PHPUnit

PHPUnit es el framework de pruebas más utilizado en el ecosistema PHP y constituye la base para la ejecución de pruebas automatizadas, incluidas las pruebas de contrato implementadas con Pact PHP. Bergmann (2024), creador de PHPUnit, señala que este framework proporciona una infraestructura robusta para organizar casos de prueba, automatizar verificaciones y generar reportes detallados de ejecución, convirtiéndose en el estándar de facto para las pruebas en PHP (Bergmann, 2024).

Pest

Pest es un framework moderno para pruebas en PHP construido sobre PHPUnit, diseñado para ofrecer una sintaxis más sencilla y legible. Según Nuno Maduro (2025), Pest mantiene toda la potencia de PHPUnit, pero simplifica la escritura de pruebas mediante una sintaxis expresiva que mejora la productividad de los desarrolladores (Nuno Maduro, 2025).

Aunque Pest no implementa pruebas de contrato por sí mismo, puede utilizarse conjuntamente con Pact PHP para ejecutar las pruebas encargadas de generar y verificar los contratos entre consumidores y proveedores.

Java

La herramienta más utilizada para implementar Contract Testing es Pact JVM, la implementación oficial del framework Pact para la plataforma Java. De acuerdo con Holshausen (2020), Pact JVM adopta el enfoque Consumer-Driven Contract Testing (CDCT), permitiendo que los consumidores definan contratos ejecutables que posteriormente son verificados por los proveedores antes de publicar nuevas versiones de sus servicios.

JUnit

JUnit es el framework de pruebas más utilizado en el ecosistema Java y constituye el entorno habitual para desarrollar y ejecutar pruebas de contrato mediante Pact JVM. Gamma y Beck (2004) señalan que JUnit introdujo el modelo xUnit en Java, proporcionando una estructura organizada para definir casos de prueba, automatizar verificaciones y generar resultados reproducibles (Beck & Gamma, 2004).

TestNG

Es otro framework ampliamente utilizado para pruebas automatizadas en Java. TestNG fue diseñado para ampliar las capacidades de JUnit, incorporando funcionalidades como ejecución paralela, agrupación de pruebas, parametrización avanzada y una configuración más flexible (Beust, 2025).

Aunque TestNG no implementa pruebas de contrato de manera nativa, puede utilizarse junto con Pact JVM para ejecutar las pruebas encargadas de generar y verificar los contratos entre consumidores y proveedores.

Selenium

Selenium es una herramienta ampliamente utilizada para automatizar pruebas funcionales sobre interfaces web. Sin embargo, su propósito es diferente al de las pruebas de contrato. Mientras Pact JVM verifica la compatibilidad de las interfaces de comunicación entre servicios, Selenium automatiza la interacción con aplicaciones web desde la perspectiva del usuario final. Por ello, ambas herramientas pueden complementarse dentro de una estrategia integral de aseguramiento de la calidad, pero cumplen funciones distintas (Selenium Project, 2025).

C# / .NET

En el ecosistema .NET, la implementación de Contract Testing se realiza principalmente mediante PactNet, la implementación oficial del framework Pact para C# y .NET. De acuerdo con Holshausen (2020), PactNet aplica el enfoque Consumer-Driven Contract Testing (CDCT), permitiendo que el consumidor defina contratos ejecutables que posteriormente son verificados por el proveedor para asegurar el cumplimiento de las interacciones acordadas (Holshausen, 2020).

NUnit

NUnit es uno de los frameworks de pruebas más utilizados en el ecosistema .NET. Según el proyecto NUnit (2025), proporciona una infraestructura robusta para crear, organizar y ejecutar pruebas automatizadas mediante atributos, aserciones y ejecución de suites de prueba (NUnit Project, 2025).

xUnit

xUnit.net es un framework moderno de pruebas para .NET diseñado como sucesor de NUnit. De acuerdo con los desarrolladores de xUnit.net (2025), este framework incorpora mejoras en el aislamiento de pruebas, ejecución paralela y facilidad de integración con herramientas modernas de desarrollo (xUnit.net, 2025).

MSTest

MSTest es el framework oficial de pruebas desarrollado por Microsoft para la plataforma .NET. Este permite crear pruebas unitarias e integrarlas directamente con Visual Studio y Azure DevOps, facilitando la automatización y administración del ciclo de pruebas (Microsoft, 2025).

Estrategia de Automatización

Las pruebas de contrato obtienen su verdadero valor de negocio al integrarse por completo en el flujo automatizado de entrega de software (Pactflow, 2024).

Automatización mediante CI/CD (Pipelines)

La verificación se automatiza e integra en herramientas de Integración y Despliegue Continuo como GitHub Actions, GitLab CI o Jenkins, operando mediante dos pipelines independientes coordinados por un bróker central (Micallef, 2021):

- Pipeline del Consumidor: Cada vez que se realiza un cambio de código (Push) en el repositorio del cliente, el pipeline ejecuta las pruebas unitarias de contrato. Si el resultado es exitoso, genera automáticamente un archivo de intercambio (usualmente en formato JSON) y lo publica en el repositorio central de contratos o Pact Broker (Fowler, 2011).
- Pipeline del Proveedor: Al activarse cualquier cambio en el repositorio del backend, su pipeline descarga de forma automática el contrato correspondiente desde el Pact Broker, lo ejecuta contra el código actual de la API y devuelve el resultado de la verificación (Aprobado o Fallido) al Bróker (Micallef, 2021).
- Compuerta de Despliegue: Antes de autorizar el paso a producción de cualquier componente, el pipeline invoca una herramienta de validación cruzada (como el comando can-i-deploy). Si el Pact Broker indica que las versiones específicas del consumidor y del proveedor no son compatibles o no han sido verificadas entre sí con

éxito, el pipeline bloquea el despliegue de forma automática, actuando como un filtro de seguridad (Pactflow, 2024).

¿Qué ocurre cuando una prueba falla y cómo se gestionan esas fallas?

Cuando se detecta una incompatibilidad o ruptura en el contrato durante la ejecución del pipeline, se activa el siguiente protocolo de gestión de fallas:

- **Bloqueo Inmediato (Build Failure):** El pipeline detiene su ejecución inmediatamente y marca la tarea como fallida. Esto actúa como un escudo que impide que el código defectuoso o incompatible progrese a ramas principales o a entornos productivos (Humphrey, 2022).
- **Aislamiento y Diagnóstico:** Al ejecutarse de forma aislada, el pipeline genera un reporte explícito del error sin necesidad de revisar múltiples servidores o bases de datos descentralizadas. Esto permite determinar de forma inmediata qué estructura o tipo de dato causó la incompatibilidad (Micallef, 2021).
- **Gestión y Resolución del Conflicto:** Al ser un fallo de acoplamiento mutuo, los equipos involucrados coordinan la solución bajo dos escenarios posibles (Humphrey, 2022):
 - Si el cambio en el backend fue un error accidental (regresión), el equipo de desarrollo del proveedor corrige el código de la API para volver al estándar acordado.
 - Si el cambio responde a una evolución planificada del sistema, el equipo del consumidor actualiza su prueba para reflejar la nueva estructura, generando un nuevo contrato y restableciendo la compatibilidad en el pipeline.

Conclusiones

Las Pruebas de Contrato constituyen una estrategia fundamental para garantizar la compatibilidad entre consumidores y proveedores en arquitecturas basadas en microservicios y APIs, permitiendo detectar incompatibilidades de forma temprana y reduciendo los riesgos durante la integración y el despliegue de nuevas versiones.

El análisis de normas y estándares internacionales, como ISO/IEC 25010, ISO/IEC/IEEE 29119, ISO/IEC/IEEE 12207 e ISTQB, evidencia que el Contract Testing se encuentra alineado con las buenas prácticas de aseguramiento de la calidad del software y contribuye al desarrollo de sistemas más confiables, mantenibles e interoperables.

El ecosistema de herramientas para Contract Testing mantiene un enfoque unificado mediante las implementaciones oficiales de Pact (Pact JS, Pact Python, Pact PHP, Pact JVM y PactNet), las cuales comparten la misma metodología de trabajo y se diferencian principalmente por su integración con los frameworks de pruebas propios de cada lenguaje de programación.

La incorporación de las Pruebas de Contrato dentro de metodologías Agile, DevOps y procesos de Integración y Entrega Continua (CI/CD) fortalece la automatización del proceso de pruebas, favorece el desarrollo independiente de los servicios y mejora la calidad del software al prevenir errores de comunicación antes de llegar a los entornos de producción.

Bibliografía

- Beck, K., & Gamma, E. (2004). *JUnit Cookbook*. O'Reilly Media.
- Bergmann, S. (2024). *PHPUnit Manual*.
- Beust, C. (2025). *TestNG Documentation*.
- Fowler, M. (2014). *Microservice Trade-Offs*.
- Holshausen, R. (2020). *Consumer Driven Contracts: A Service Evolution Pattern*. Pact Foundation.
- International Software Testing Qualification Board. (2023). *Certified Tester Foundation Level Syllabus Version 4.0*.
- Meta Open Source. (2025). *Jest Documentation*.

Microsoft. (2025). *MSTest framework documentation*.

Mocha.js. (2025). *Mocha Documentation*.

NUnit Project. (2025). *NUnit Documentation*.

Nuno Maduro. (2025). *Pest Documentation*.

Okken, B. (2022). *Python Testing with pytest (2nd ed.)*.

Pact Foundation. (2025). *Pact JS Documentation*.

Python Software Foundation. (2025). *unittest — Unit testing framework*.

Richardson, C. (2021). *Microservices Patterns*.

Selenium Project. (2025). *Selenium Documentation*.

xUnit.net. (2025). *xUnit.net Documentation*.